



Welcome to module 2

Welcome back! I'm excited to continue our learning journey together. I really enjoyed exploring the basics of Python programming with you. You learned that Python is a powerful tool for data professionals, and lets you analyze data with speed, accuracy, and efficiency. You now know how to use variables in data types to store and organize your data, and have already started writing your own Python code and this course will continue to build on your foundation of Python knowledge. By the end of the course, you'll be able to write statements in Python code to perform multi-step operations on your data. You'll also learn how to write clean, readable code that can be easily understood and reused by other data professionals.

Being able to collaborate with teammates is one of the most important skills for a data professional. And, writing clean code is a great way to collaborate with your teammates and help your team achieve its goals. Working with clean code helps your team work faster, communicate more effectively, and produce better results. We'll start with functions, or reusable chunks of code, that let you perform specific tasks. Functions are like the verbs or action words of a programming language. You can call on a function at any time to help you perform useful actions on your data, such as sorting, classifying,

summarizing, and much more. Then we'll discuss how to write clean code that can be easily understood by your teammates and collaborators.

You'll learn about two important elements of writing clean code: reusability and modularity. Both of these practices speed up project development and help data professionals focus on core business needs and avoid spending time doing rework.

After that, we'll examine another key aspect of writing clean code: commenting.

Commenting is a useful practice, because it helps you consider your thought process while documenting your workflow for teammates. Using comments to describe the component parts of a problem helps you solve it in clear, simple steps. Next, we'll discuss how to use operators to compare values. We'll review two types of operators: comparators and logical operators.

Comparators such as greater than or less than allow you to compare two values.

Logical operators such as AND and OR let you connect multiple statements together and perform more complex comparisons. Data professionals use operators to analyze and make decisions about their data. Lastly, we'll consider conditional statements, which tell the computer how to make decisions based on your instructions. You'll learn how to write if, else, and else-if statements. Data professionals use conditional statements to structure complex operations, and to perform all kinds of practical tasks, such as binning data and organizing files. Conditional statements make your Python code more flexible and powerful.

When you're ready, I'll meet you in the next video.

What you'll learn

- Functions
- Write clean code
- Commenting
- Operators
- Conditional statements